Aprendizaje automático avanzado
Codificación por contexto

Juan Miguel Santos

Laboratorio de investigación y desarrollo experimental en computación (LIDEC)
Universidad Nacional de Hurlingham

(13 de agosto de 2026)

Procesamiento de lenguaje natural

Introducción

- Procesamiento de lenguaje natural es la disciplina que se centra en el desarrollo y estudio de sistemas automáticos para la:
  - interpretación de lenguajes naturales, y
  - generación de lenguajes naturales.
- Se lo nota como PLN o *NLP* del término en inglés *Natural Language Processing*.

Procesamiento de lenguaje natural

Introducción

- Como lenguaje natural se entiende al lenguaje que usan **los humanos para comunicarse entre sí** desde los inicios de su vida.

Procesamiento de lenguaje natural

Algunas aplicaciones de PLN

- Traducción automática.
- Respuesta a preguntas o recuperación de la información.
- Resumen y análisis de texto.
- Reconocimiento del habla o de imágenes.

Procesamiento de lenguaje natural

Algunas aplicaciones de PLN

- La **traducción automática** se la abrevia como TA (*MT* del término en inglés *Machine Translation*). Es tema de estudio desde hace varias décadas.
- Uno de los problemas que presenta es cuando es necesario traducir textos muy largos.
- Por ejemplo, cuando la semántica de un fragmento o una palabra fue dada al principio de un texto extenso y se alude a dicha semántica al final del mismo.

Procesamiento de lenguaje natural

Algunas aplicaciones de PLN

- Pero no es el único.
- ¿Cuándo dos palabras que se escriben diferente refieren a lo mismo (sinónimos o el uso de pronombres, por ejemplo)?

Procesamiento de lenguaje natural

Algunas aplicaciones de PLN

- **Respuesta a preguntas o recuperación de la información** es típico de una búsqueda en la web o aplicaciones de chats automáticos.
- Por ejemplo:
  - ¿Cómo puedo limpiar mi sartén de teflón?
  - ¿Cuál es la ciudad con mayor expectativa de vida en la Argentina?
  - Me gustaría saber qué ocurrió con mi pedido que no lo recibí,
  - ¿Cómo puedo solucionar el error de compilación en el código python que cargué?

Procesamiento de lenguaje natural

Algunas aplicaciones de PLN

- **Resumen y análisis de texto** puede ser usado con múltiples propósitos.
- Por ejemplo:
  - conocer opiniones de los usuarios en una red social,
  - describir los principales conceptos vertidos en un texto,
  - describir un esquema de un procedimiento en pasos a seguir,
  - etcétera.

Procesamiento de lenguaje natural

Algunas aplicaciones de PLN

- **Reconocimiento del habla o de las imágenes** consiste en transcribir una conversación o monólogo de audio en un texto escrito o en obtener una descripción escrita de una imagen o video.
- Tanto en una dirección como en la contraria.

Procesamiento de lenguaje natural

Representación

- Un aspecto fundamental de construir máquinas que procesen adecuadamente un lenguaje natural es el tema de la representación.
- La forma en que escribimos y hablamos sigue una codificación no numérica tales como letras y fonemas.
- Si deseamos tratar a los lenguajes con máquinas (computadoras) es necesario encontrar una forma de pasar de fonemas o letras a representaciones numéricas, y viceversa.

Representación de las palabras

Introducción

¿Qué es una palabra?

- Es una cadena de símbolos de un alfabeto finito.
- Nosotros no usamos cadenas arbitrarias de símbolos sino que un subconjunto de todas las posibles combinaciones.
- Por ejemplo, no usamos las palabras tragerta o benfoitema pero sí usamos tabla, juego, etc.

Representación de las palabras

Introducción

¿Qué es un token?

- Es una instancia de un fragmento de texto en un contexto dado.
- Por ejemplo, en

  Mi casa linda con la de la familia Pascuali,

  los *tokens* podrían ser {Mi, casa, linda, con, la, de, la, familia Pascuali}
- Fijarse hay dos *tokens* para la palabra "la".

Representación de las palabras

Introducción

¿Qué es un token?

- Pero también los *tokens* podrían ser letras: {M, i, c, a, s, l, n, d, o, e, f, m, P, u}.
- Podrían ser sílabas o fragmentos de palabras.

Representación de las palabras

Introducción

¿Qué es un token?

- Hay criterios para determinar cuáles son los *tokens* más convenientes a utilizar.
- Por el momento, los tokens que vamos a considerar nosotros serán las palabras.

Representación de las palabras

one-hot

- Una posible representación de palabras es mediante vectores.
- Por ejemplo, si tengo un diccionario de 10 palabras {ojo, sal, mío, ala, mesa, silla, sol, madera, agua, luz} puedo representar dichas palabras usando vectores de dimensión 10.
- Este tipo de representación se denomina *one-hot*.

Representación de las palabras

one-hot

Dado el diccionario
{ojo, sal, mío, ala, mesa, silla, sol, madera, agua, luz}
se puede representar las palabras como:

para **ojo** uso el vector (1, 0, 0, 0, 0, 0, 0, 0, 0, 0)
para **sal** uso el vector (0, 1, 0, 0, 0, 0, 0, 0, 0, 0)
para **mío** uso el vector (0, 0, 1, 0, 0, 0, 0, 0, 0, 0)
...
para **luz** uso el vector (0, 0, 0, 0, 0, 0, 0, 0, 0, 1)

Representación de las palabras

one-hot

- Como son vectores, podría usar el *producto interno* para saber si las palabras son *parecidas* entre sí.
- Nota: recordar que cuanto más alineados están dos vectores mayor es el valor de su producto interno. Y cuando más alineados están más se parecen sus componentes.
- Sin embargo, el resultado para este representación es que todas las palabras son distintas entre sí (todos los productos internos dan 0), lo cual es cierto.

Representación de las palabras

one-hot

- Sabemos que 'sol' y 'luz' tienen *algo en común* o al menos, a simple vista, tienen más en común que 'sol' y 'sal'.
- Es decir, estamos perdiendo algo: esta representación no permite expresar la similitud o disimilitud entre las palabras .

Representación de las palabras

one-hot

- Y no solo eso. La representación o codificación que usamos **no nos dice nada acerca del significado de las palabras**.
- Esto último es importante que lo tomemos en cuenta **durante todo el curso** y para todo los tipos de representaciones que consideremos.

Representación de las palabras

Representación por aspectos lingüisticos

- Otra alternativa es usar una representación que tenga en cuenta aspectos lingüísticos y semánticos de cada palabra.
- Por ejemplo, en vez de pensar en un vector de 0s con un 1 en un lugar específico, se podría pensar que cada elemento del vector representa una **característica lingüística** o **semántica**.

Representación de las palabras

Representación por aspectos lingüisticos

Por ejemplo:

- ¿Es una palabra en singular?
- ¿Es género femenino, masculino o neutro?
- ¿Es un sustantivo?
- ¿Es un verbo?
- ¿Es tercera persona?
- ¿Es un ser vivo?
- ¿Es humano?
- etc.

Representación de las palabras

Representación por aspectos lingüisticos

- Un problema de esta representación es elegir las características adecuadas.
- Otro problema, es completar la información para la representación de todas las palabras de un diccionario.
- También, esto solo se podría aplicar a palabras. ¿Qué pasaría si nuestros *tokens* son fragmentos de palabras?

Representación de las palabras

Representación por co-ocurrencia

- Una forma alternativa de representar palabras consiste en representar una palabra de acuerdo al entorno de la misma.
- Para hacer esto tendremos en cuenta que vamos a usar varios documentos.
- Por ejemplo, consideremos los documentos de un portal periodístico (donde cada documento será una nota, crónica, editorial o columna periodística).

Representación de las palabras

Representación por co-ocurrencia

- Consideremos una matriz que tiene una columna por cada palabra del diccionario. Lo mismo para las filas.
- Consideremos la palabra p y contamos cuántas veces aparece cada palabra p' del diccionario en los documentos donde aparece p.
- Ingresamos esa cantidad en la casilla (p, p').
- Finalmente normalizamos la fila dividiendo por la suma de las cantidades en la misma.

Representación de las palabras

Representación por co-ocurrencia

- Este método se denomina representación por matriz de co-ocurrencia.
- Se puede hacer también, contando las apariciones de p' en el documento solo si p' está en un entorno o **contexto** de p.
- Esta representación nos da más información que *on-hot*.

Representación de las palabras

Representación por co-ocurrencia

- De todos modos, al igual que *on-hot* necesitaríamos para cada palabra un vector cuya dimensión es igual a la cantidad de palabras del diccionario (por ejemplo, un vector de dimensión 50000).

Representación de las palabras

Método *word2vec*

- El método *word2vec* representa cada palabra usando un vector que tiene una dimensión mucho menor que la cantidad de palabras ($|V|$) de un diccionario V. Por ejemplo, usando vectores de dimensión 300.

Representación de las palabras

Método *word2vec*

- La idea de este método es representar las palabras con un vector en los reales de modo tal que
  si una palabra p **comparte frecuentemente el mismo contexto** con p'
  entonces
  p debería tener una **representación más similar** a p' que a otra palabra q con la cual no comparte frecuentemente contexto.

Representación de las palabras

Método *word2vec*

- Veamos cómo podemos escribir esta idea de una manera más formal.
- Sea un diccionario V donde $o, c \in V$, y sean v_o y v_c los vectores que las representan, respectivamente.
- Queremos expresar que si la palabra o está frecuentemente en el entorno de c, entonces la representación de o y c deberían ser similares, o dicho de otro modo, el producto interno entre v_o y v_c debería ser significativo.

Representación de las palabras

Método *word2vec*

- Esta relación la podemos expresar como la probabilidad que o esté en el entorno de c está dado por

$$P(o/c) = \frac{\exp^{v_o^t v_c}}{\sum_{p \in V} \exp^{v_p^t v_c}}.$$

Representación de las palabras

Método *word2vec*

- Fijarse que en la ecuación anterior, $\exp^{v_o^t v_c}$ será alta si v_o y v_c se parecen, es decir si su producto interno es alto también.
- Como o está en el entorno de c, la $P(o/c)$ debería ser alta.
- Luego, estamos buscando representaciones de o y de c tales que si $P(o/c)$ es alta, v_o y v_c sea similares.

Representación de las palabras

Método *word2vec*

- Podríamos plantear el problema así:
  ¿Cómo debemos representar (qué vectores asignarles) cada palabra tal que si para cada par de palabras p y p' cualesquiera que comparten el contexto (o sea, la probabilidad de $P(p'/p)$ es alta), el producto interno entre v_p y $v_{p'}$ sea alto también?

Representación de las palabras

Método *word2vec*

- Sea un *corpus* formado por un conjunto de T documentos y sea $|D_t|$ la cantidad de palabras del documento D_t con $1 \leq t \leq T$.
- Sea m el tamaño del contexto a izquierda y a derecha de una palabra p en un documento D_t.

Representación de las palabras

Método *word2vec*

- Podemos plantear la siguiente función a maximizar:

$$L(U) = \prod_{t=1}^{T} \prod_{j=m+1}^{|D_t|-m} \prod_{i=-m, i\neq 0}^{m} P(p_{j+i}/p_j, U)$$

donde $U \in R^{|V|\times d}$ es el conjunto de las representaciones v_p de las palabras p del diccionario V y donde cada representación v_p es una fila de U de dimensión d.

Representación de las palabras

Método *word2vec*

- Si juntamos todos los documentos del corpus en un solo documento, podemos expresar la ecuación anterior como:

$$L(U) = \prod_{j=m+1}^{|D|-m} \prod_{\substack{i=-m, \\ i \neq 0}}^{m} P(p_{j+i}/p_j, U)$$

donde D es el único documento que concatena todos los documentos del corpus.

Representación de las palabras

Método *word2vec*

- La función $L(U)$ que deseamos maximizar la podemos expresar como una función $J(U)$ para minimizar:

$$J(U) = -\frac{1}{|D|} logL(U) = -\frac{1}{|D|} \sum_{j=m+1}^{|D|-m} \sum_{\substack{i=-m, \\ i \neq 0}}^{m} \log P(p_{j+i}/p_j, U).$$

Representación de las palabras

Método *word2vec*

- Es decir, queremos encontrar los valores de U tal que $J(U)$ sea mínima o $L(U)$ sea máxima.
- Esto último, tal como mencionamos antes, es obtener las representaciones de cada palabra en V como vectores de dimensión d tal que
  el producto interno entre dos vectores v_p y v'_p correspondientes a dos palabras cualesquiera p y p' en V, sea alto si dichas palabras suelen compartir contextos de tamaño $2m$.

Representación de las palabras

Método *word2vec*

- Para eso vamos a derivar $J(U)$ respecto a las componentes de U, es decir los vectores de representación v.

Representación de las palabras

Método *word2vec*

- Antes de proceder haremos un pequeño cambio en la notación. Dado un contexto de longitud $2m+1$, es decir, formado por $2m+1$ palabras, llamaremos v_c a la representación de palabra central mientras que usaremos v_o para notar a las representaciones de las palabras alrededor de la palabra central y que no están a una distancia mayor a m.

Representación de las palabras

Método *word2vec*

- Observamos que $J(U)$ es $-\frac{1}{|D|}$ por la sumatoria doble de términos del estilo:

$$\log P(o/c, U)$$

  por lo tanto nos concentraremos en derivar dichos términos respecto a v_c.
- Como habíamos visto

$$\log P(o/c, U) = \log(exp^{(v_o^t v_c)} / \sum_{p \in V} exp^{(v_p^t v_c)}).$$

Representación de las palabras

Método *word2vec*

- Así, la $\partial \log P(o/c, U)/\partial v_c$ es

$$\partial \log(exp^{(v_o^t v_c)} / \sum_{p \in V} exp^{(v_p^t v_c)})/\partial v_c =$$

$$\partial \log exp^{(v_o^t v_c)}/\partial v_c - \partial log \sum_{p \in V} exp^{(v_p^t v_c)}/\partial v_c.$$

.

Representación de las palabras

Método *word2vec*

- Por un lado,
$$\partial \log exp^{(v_o^t v_c)} / \partial v_c = \partial (v_o^t v_c) / \partial v_c = v_o.$$

Representación de las palabras

Método *word2vec* (nota)

Fijarse que:

- La $\partial f/\partial v$, con v vector, $v = (v_1, v_2, ..., v_n)$, es el vector $\partial f/\partial v_1, \partial f/\partial v_2, ..., \partial f/\partial v_n$, donde n es la dimensión de v.
- Ahora consideremos la función f como el producto de dos vectores

  $$f = u^t v = u_1 * v_1 + ... + u_n * v_n.$$

  Así, la $\partial f/\partial v$ es

  $$(\partial f/\partial v_1, \partial f/\partial v_2, ..., \partial f/\partial v_n) = (u_1, u_2, ..., u_n) = u.$$

Representación de las palabras

Método *word2vec*

- Por otro lado, tenemos $\partial \log \sum_{p \in V} exp^{(u_p^t v_c)} / \partial v_c$.
- Dado que la derivada de $\log(x)$ es $\frac{1}{x}$ podemos expresar $\partial \log \sum_{p \in V} exp^{(u_p^t v_c)} / \partial v_c$ como:

$$\frac{1}{\sum_{p \in V} exp^{(u_p^t v_c)}} \partial \sum_{x \in V} exp^{(u_x^t v_c)} / \partial v_c =$$

$$\frac{1}{\sum_{p \in V} exp^{(u_p^t v_c)}} \sum_{x \in V} \partial (exp^{(u_x^t v_c)} / \partial v_c) =$$

$$\frac{1}{\sum_{p \in V} exp^{(u_p^t v_c)}} \sum_{x \in V} exp^{(u_x^t v_c)} \partial (u_x^t v_c) / \partial v_c) =$$

$$\frac{1}{\sum_{p \in V} exp^{(u_p^t v_c)}} \sum_{x \in V} exp^{(u_x^t v_c)} u_x.$$

Representación de las palabras

Método *word2vec*. Recapitulando

- Teníamos que la $\partial \log P(o/c, U)/\partial v_c$ es
  $\partial \log exp^{(v_o^t v_c)}/\partial v_c - \partial \log \sum_{p \in V} exp^{(v_p^t v_c)}/\partial v_c$.
- Según vimos $\partial \log exp^{(v_o^t v_c)}/\partial v_c$ es v_o
  y
  $\partial \log \sum_{p \in V} exp^{(v_p^t v_c)}/\partial v_c$ es
  $$\frac{1}{\sum_{p \in V} exp^{(v_p^t v_c)}} \sum_{x \in V} exp^{(v_x^t v_c)} v_x = \sum_{x \in V} \frac{exp^{(v_x^t v_c)}}{\sum_{p \in V} exp^{(v_p^t v_c)}} v_x.$$

Representación de las palabras

Método *word2vec*. Recapitulando

- Así que

$$\partial \log P(o/c, U)/\partial v_c = v_o - \sum_{x \in V} \frac{exp^{(v_x^t v_c)}}{\sum_{p \in V} exp^{(v_p^t v_c)}} v_x.$$

Representación de las palabras

Método *word2vec*. Concluyendo

- Pero $\frac{exp^{(v_x^t v_c)}}{\sum_{p \in V} exp^{(v_p^t v_c)}}$ es $P(x/c)$ de modo que

$$\partial \log P(o/c, U)/\partial v_c = v_o - \sum_{x \in V} p(x/c) v_x.$$

Representación de las palabras

Método *word2vec*. Concluyendo

- Para completar la derivada parcial de $J(U)$ respecto a v_c, recordemos que $J(U)$ es:

$$J(U) = -\frac{1}{|D|} logL(U) = -\frac{1}{|D|} \sum_{j=m+1}^{|D|-m} \sum_{\substack{i=-m, \\ i \neq 0}}^{m} \log P(p_{j+i}/p_j, U).$$

Representación de las palabras

Método *word2vec*. Concluyendo

- La doble sumatoria es para poder sumar todas las posibilidades en el documento donde se da una palabra central c y su entorno de tamaño m donde están las palabras o. Si derivamos respecto a un v_c particular, nos queda que el gradiente respecto a v_c será:

$$-\frac{1}{|D|}\partial \log P(o/c, U)/\partial v_c = -\frac{1}{|D|}v_o + \frac{1}{|D|}\sum_{x \in V} p(x/c)v_x.$$

Representación de las palabras

Método *word2vec*. Concluyendo

- Así, podemos actualizar el valor de v_c usando el método del gradiente descendente:

$$v_c^{nuevo} = v_c^{anterior} + \eta(v_o - \sum_{x \in V} p(x/c)v_x).$$

  donde η incluye a $\frac{1}{|D|}$ y el signo $+$ aparece al multiplicar por -1 al gradiente.
- Podemos realizar el mismo procedimiento para obtener una regla de actualización para v_o.